

# RestClient

## RestClient 기본 사용법

Spring 6.1 (Spring Boot 3.2+)에서 도입된 현대적 동기 **HTTP** 클라이언트다.  
RestTemplate의 단순함 + WebClient의 깔끔한 API를 합친 것이다.  
이 프로젝트에서 사용 중인 **HTTP** 클라이언트다.

## 왜 RestClient인가

RestTemplate의 문제:

- 헤더를 추가하려면 `HttpEntity`, `exchange()` 등 장황한 코드가 필요
- 유지보수 모드 (새 기능 추가 없음)

WebClient의 문제:

- Spring MVC에서는 `.block()`으로 동기 전환해야 함 → 비동기 이점 없음
- WebFlux 의존성 필요
- 리액티브 학습 곡선

RestClient:

- WebClient처럼 깔끔한 체이닝 API
- 동기 방식이라 Spring MVC에 자연스러움
- 별도 의존성 불필요 (spring-web에 포함)
- Spring 공식 권장

요약하면, 동기 호출이면서 모던한 **API**를 원할 때 선택하는 클라이언트가 RestClient 다.

## 1. 생성 방법

### 기본 생성

```
// 가장 단순한 생성
RestClient restClient = RestClient.create();

// baseUrl 지정
RestClient restClient = RestClient.create("https://api.example.com");
```

## Builder로 생성 (권장)

```
// 이 프로젝트의 공통 패턴
HttpClient httpClient = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(3))
    .build();

JdkClientHttpRequestFactory factory = new
JdkClientHttpRequestFactory(httpClient);
factory.setReadTimeout(Duration.ofSeconds(5));

RestClient restClient = RestClient.builder()
    .requestFactory(factory)
    .build();
```

## 기존 RestTemplate에서 생성

```
// 레거시 프로젝트에서 점진적으로 마이그레이션할 때
RestClient restClient = RestClient.create(existingRestTemplate);
```

기존 RestTemplate 의 설정(메시지 컨버터, 인터셉터, 요청 팩토리 등)을 그대로 물려받으므로 점진적 전환에 유용하다.

## 2. GET 요청

### 기본 GET

```
String result = restClient.get()
    .uri("https://api.example.com/data")
    .retrieve()
    .body(String.class);
```

### 헤더 포함 GET

```
// 프로젝트 실제 코드: NaverNewsClient.java
NaverNewsResponse response = restClient.get()
    .uri(uri)
    .header("X-Naver-Client-Id", clientId)
    .header("X-Naver-Client-Secret", clientSecret)
    .retrieve()
    .body(NaverNewsResponse.class);
```

### URI 변수 사용

```
// Path Variable
MemberDetail member = restClient.get()
    .uri("https://api.example.com/members/{id}", 1L)
    .retrieve()
    .body(MemberDetail.class);

// 여러 변수
restClient.get()
    .uri("https://api.example.com/orgs/{orgId}/members/{memberId}", orgId,
memberId)
    .retrieve()
    .body(MemberDetail.class);
```

{id} 같은 URI 템플릿 변수는 자동으로 URL 인코딩되므로, 직접 문자열을 이어붙이는 것보다 안전하다.

## UriBuilder로 쿼리 파라미터 조합

```
// 프로젝트 실제 패턴: NaverNewsClient.java
NaverNewsResponse response = restClient.get()
    .uri(uriBuilder -> uriBuilder
        .scheme("https")
        .host("openapi.naver.com")
        .path("/v1/search/news.json")
        .queryParams("query", keyword)
        .queryParams("display", 10)
        .queryParams("sort", "date")
        .build())
    .header("X-Naver-Client-Id", clientId)
    .header("X-Naver-Client-Secret", clientSecret)
    .retrieve()
    .body(NaverNewsResponse.class);
```

## 3. POST 요청

### JSON Body 전송

```
ResponseCreateMember response = restClient.post()
    .uri("https://api.example.com/members")
    .contentType(MediaType.APPLICATION_JSON)
    .body(new RequestCreateMember("홍길동"))
    .retrieve()
    .body(ResponseCreateMember.class);
```

## 프로젝트 실제 코드: Discord 웹훅

```
// DiscordWebhookClient.java
restClient.post()
    .uri(webhookUrl)
    .contentType(MediaType.APPLICATION_JSON)
    .body(new DiscordMessage(content))
    .retrieve()
    .toBodilessEntity(); // 응답 body가 없는 경우
```

---

## 4. PUT / DELETE 요청

```
// PUT
restClient.put()
    .uri("https://api.example.com/members/{id}", 1L)
    .contentType(MediaType.APPLICATION_JSON)
    .body(updateRequest)
    .retrieve()
    .body(ResponseUpdateMember.class);

// DELETE
restClient.delete()
    .uri("https://api.example.com/members/{id}", 1L)
    .retrieve()
    .toBodilessEntity();
```

---

## 5. 응답 처리 방식

### body() - 응답 Body만 필요할 때

```
NaverNewsResponse body = restClient.get()
    .uri(url)
    .retrieve()
    .body(NaverNewsResponse.class);
```

### toEntity() - 상태 코드, 헤더도 필요할 때

```
ResponseEntity<NaverNewsResponse> response = restClient.get()
    .uri(url)
    .retrieve()
    .toEntity(NaverNewsResponse.class);
```

```

HttpStatusCode status = response.getStatusCode();
HttpHeaders headers = response.getHeaders();
NaverNewsResponse body = response.getBody();

```

## toBodilessEntity() - 응답 Body가 없을 때

```

// 201 Created, 204 No Content 등
ResponseEntity<Void> response = restClient.post()
    .uri(url)
    .body(request)
    .retrieve()
    .toBodilessEntity();

```

## body(byte[].class) - 바이너리 응답

```

// 프로젝트 실제 코드: KasiHolidayClient.java (XML 응답)
byte[] xmlBytes = restClient.get()
    .uri(uri)
    .header(HttpHeaders.ACCEPT_CHARSET, StandardCharsets.UTF_8.name())
    .retrieve()
    .body(byte[].class);

```

retrieve() 는 기본적으로 4xx/5xx 응답에 대해 예외를 던진다. 상태 코드별로 직접 처리하고 싶다면 onStatus() 를 체이닝하거나, exchange() 를 사용해 응답을 직접 다룰 수 있다.

```

// onStatus 예시
restClient.get()
    .uri(url)
    .retrieve()
    .onStatus(HttpStatusCode::is4xxClientError, (req, res) -> {
        throw new RuntimeException("클라이언트 에러: " + res.getStatusCode());
    })
    .body(String.class);

```

## 6. 체이닝 구조 정리

```

restClient
    .get()                // 1. HTTP 메서드 선택
    .uri(url)             // 2. URL 설정
    .header("key", "value") // 3. 헤더 설정 (선택)
    .accept(MediaType.APPLICATION_JSON) // 4. Accept 헤더 (선택)

```

```
.retrieve() // 5. 요청 실행
.body(ResponseType.class); // 6. 응답 변환
```

```
get() / post() / put() / delete() / patch()
  ↓
uri()
  ↓
header() / contentType() / accept() (선택)
  ↓
body() (POST/PUT만, 선택)
  ↓
retrieve()
  ↓
body() / toEntity() / toBodilessEntity()
```

## 7. 추가 필기

### ClientHttpRequestFactory

- `RestClient`, `RestTemplate` 는 자체적으로 통신을 하지 않는다
- 내부적으로 `ClientHttpRequestFactory` 라는 추상화를 통해 실제 HTTP 통신을 위임한다
- 해당 팩토리 수준에서 연결 타임아웃, 읽기 타임아웃 등을 설정할 수 있다

`RestClient` -> `ClientHttpRequestFactory` -> 실제 HTTP 라이브러리

즉, `RestClient` 는 "어떻게 요청을 조립할지"를 담당하고, 실제 소켓 통신은 `ClientHttpRequestFactory` 구현체에 위임하는 구조다.

### ClientHttpRequestFactory 구현체

#### SimpleClientHttpRequestFactory

- `java.net.HttpURLConnection` 기반
- JDK 기본, 가장 단순하지만 기능이 빈약하다 (커넥션 풀 미지원 등)

#### JdkClientHttpRequestFactory

- JDK 11+
- `java.net.http.HttpClient` 기반
- JDK 11부터 내장된 모던 `HttpClient`
- 별도 의존성 불필요

#### HttpComponentsClientHttpRequestFactory

- Apache HttpClient 엔진 사용
- 커넥션 풀 등 기능이 풍부하다
- 별도의 의존성이 필요하다

## OkHttpClientHttpRequestFactory

- OkHttpClient 엔진 사용
- 가볍고 성능이 좋다
- 별도의 의존성이 필요하다

운영 환경에서 커넥션 풀과 세밀한 타임아웃 제어가 필요하다면 보통 `HttpComponents` (Apache HttpClient) 또는 `OkHttp` 를 선택한다.

## JdkClientHttpRequestFactory 예시

### 1) JDK HttpClient 생성 — 실제 통신 엔진

```
HttpClient httpClient = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(3)) // 연결 타임아웃은 여기서
    .build();
```

### 2) 그 엔진을 RequestFactory로 감싼다

```
JdkClientHttpRequestFactory factory
    = new JdkClientHttpRequestFactory(httpClient);

factory.setReadTimeout(Duration.ofSeconds(5)); // 읽기 타임아웃은 여기서
```

### 3) RestClient에 장착

```
RestClient client = RestClient.builder()
    .requestFactory(factory) // 이 엔진으로 통신한다
    .build();
```

- JDK HttpClient 자체는 연결 타임아웃(`connectTimeout`) 만 제공한다
- 읽기 타임아웃(`readTimeout`) 은 HttpClient 에 없으므로 factory 레벨에서 설정한다 (그래서 둘이 분리되어 있다)
- RestClient 는 별도로 지정하지 않으면 클래스패스에 있는 라이브러리를 보고 자동으로 factory 를 고른다 (예: Apache HttpClient가 있으면 그것을 사용)
- 직접 타임아웃을 설정하거나 엔진을 지정하고 싶다면, 위처럼 factory 를 직접 만들어 주입한다

## 정리

| 항목    | 내용                            |
|-------|-------------------------------|
| 도입    | Spring 6.1 / Spring Boot 3.2+ |
| 방식    | 동기 (Blocking)                 |
| 의존성   | spring-web 에 포함 (별도 불필요)      |
| 통신 위임 | ClientHttpRequestFactory 구현체  |
| 권장 상황 | Spring MVC 기반의 동기 외부 API 호출   |

RestClient 는 RestTemplate 의 단순함과 WebClient 의 깔끔한 체이닝 API를 결합한 동기 클라이언트다. 타임아웃·커넥션 풀 같은 통신 세부 설정은 ClientHttpRequestFactory 구현체에서 관리하며, 요구사항에 맞는 엔진(JDK/Apache/OkHttp)을 골라 주입하면 된다.